Based on:
- Seidl et al.'s "UML@Classroom", Chapter 4
- Miles and Hamilton's "Learning UML 2.0", Chapters 4 & 5

UML Class Diagrams

Software Design (40007) – 2025/2026

Justus Bogner
Ivano Malavolta

VRIJE UNIVERSITEIT AMSTERDAM

Software and Sustainability research group (S2)
Department of Computer Science, Faculty of Sciences

Roadmap

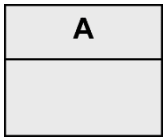
- Basics of class diagrams
- Relationships between classes
- Creating class diagrams (concrete example)

} Brief recap of UML syntax

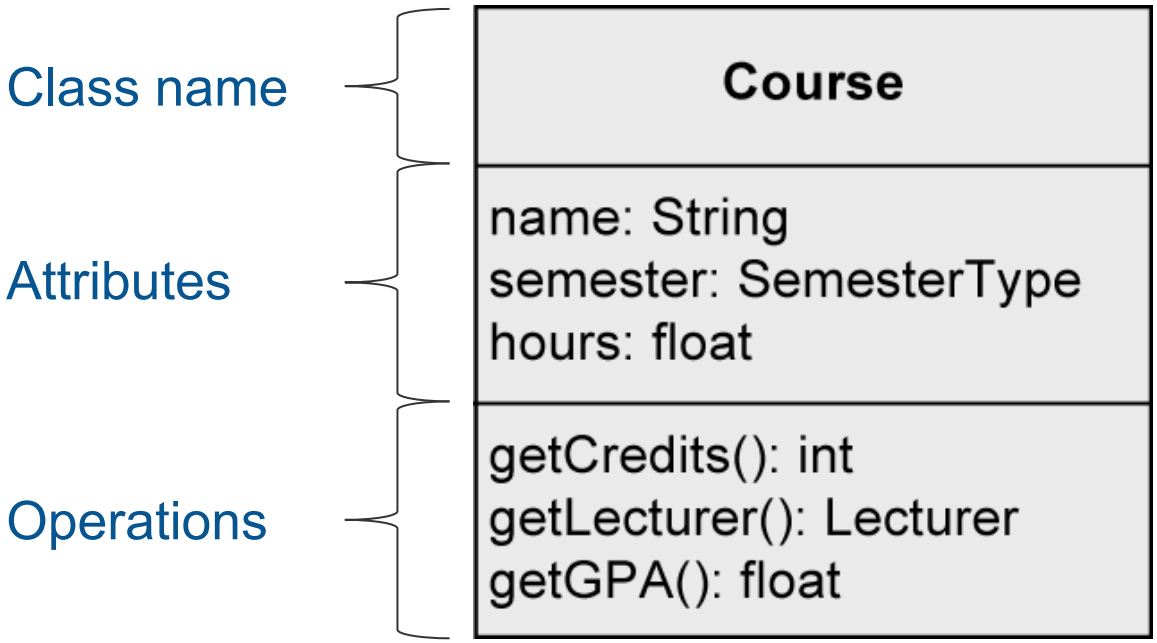
VU VRIJE UNIVERSITEIT AMSTERDAM

Basics of Class Diagrams

Class



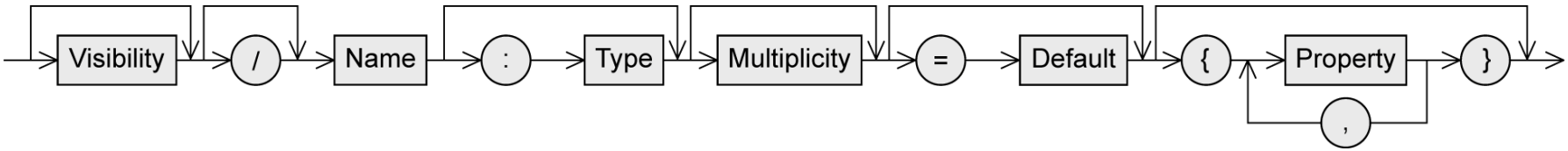
A class is a **construction plan** for a set of similar objects.



Note: in UML, an **`operation`** is the specification, while a **`method`** is the actual implementation.

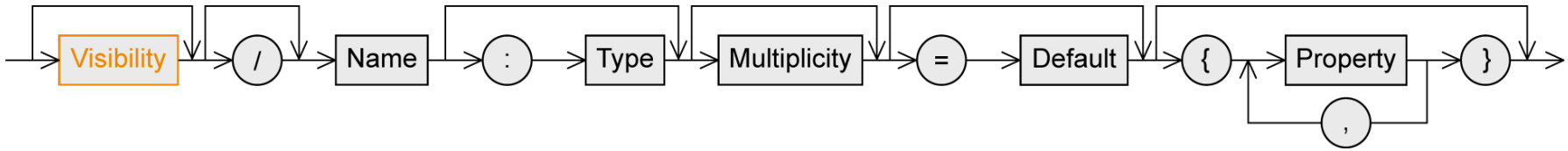
VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax



VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - visibility



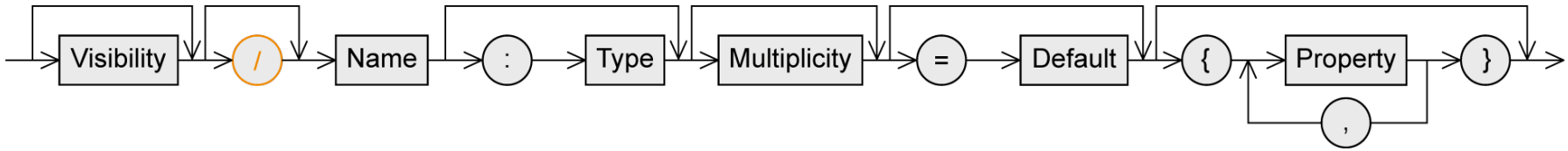
Person
+ firstName: String + lastName: String – dob: Date # address: String[1..*] {unique, ordered} – ssNo: String {readOnly} – /age: int – password: String = "pw123" – <u>personsNumber: int</u>

Who is permitted to access the attribute

- + … **public**: every other class
- - … **private**: only the class itself
- # … **protected**: class itself and subclasses
- ~ … **package**: classes in the same package

VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - derived attribute



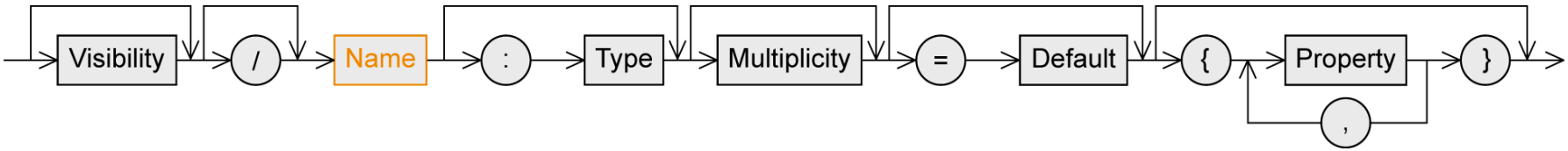
Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

Value is derived from other attributes

- **age**: calculated from the date of birth

VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - name

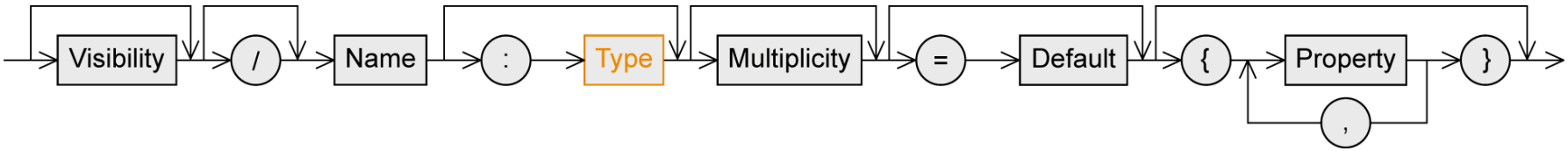


Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" personsNumber: int

Name of the attribute

VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - type



Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" personsNumber: int

Type

- Primitive data type
  - Pre-defined:
    - Boolean, Integer, String, UnlimitedNatural
  - User-defined: «**primitive**»
  - Composite data type: «**datatype**»
    - Similar to class but:
      - Instances are value objects
      - Identity defined by its properties
      - Immutable
- Enumerations: «**enumeration**»

«primitive» **Float**
round(): void

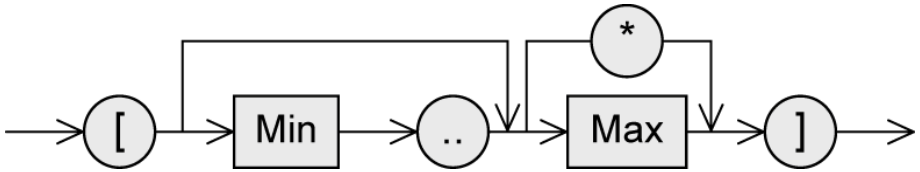
«datatype» **Date**
day month year

«enumeration» **AcademicDegree**
bachelor master phd

Attribute syntax - multiplicity

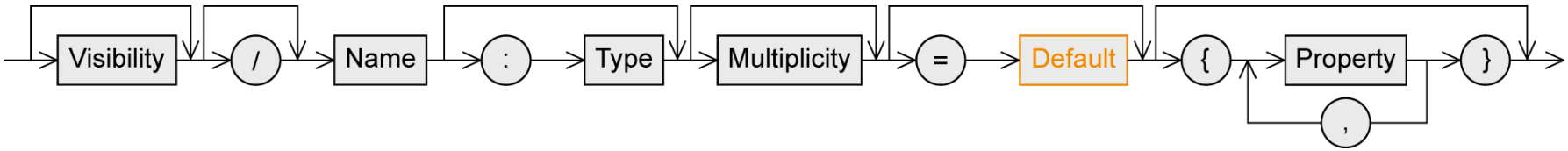
Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" personsNumber: int

- Number of values an attribute may have
- Default value: 1
- Notation: **[min..max]**
  - no upper limit: **[*]** or **[0..*]**



VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - default value



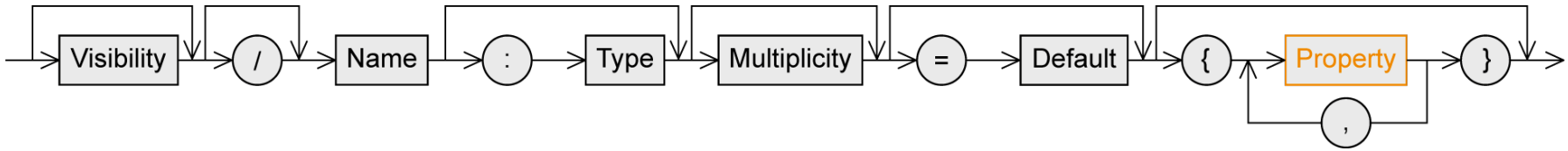
Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

Default value

- Used if the attribute value is not set explicitly by the user

VU VRIJE UNIVERSITEIT AMSTERDAM

Attribute syntax - properties

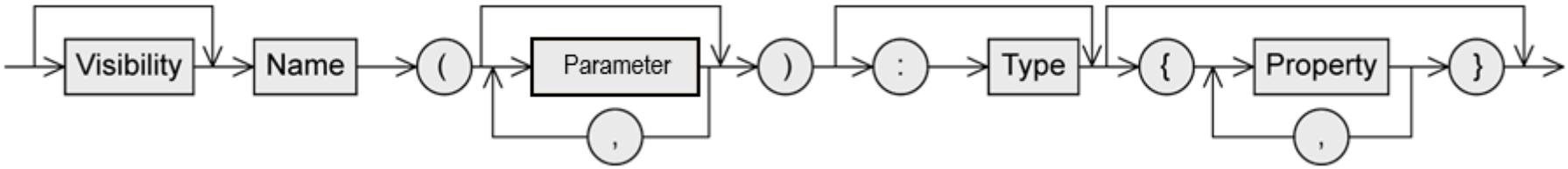


Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

- Pre-defined properties
  - {**readOnly**} … value cannot be changed
  - {**unique**} … no duplicates permitted
  - {**non-unique**} … duplicates permitted
  - {**ordered**} … fixed order of the values
  - {**unordered**} … no fixed order of the values
- Examples:
  - Set: {unordered, unique}
  - List in ascending order: {ordered, non-unique}

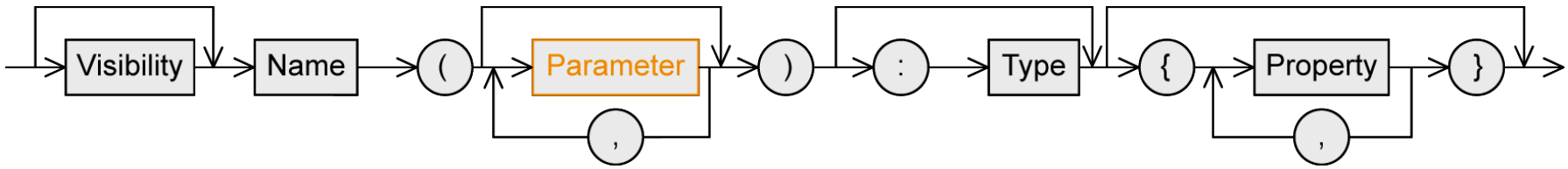
VU VRIJE UNIVERSITEIT AMSTERDAM

Operation syntax



VU VRIJE UNIVERSITEIT AMSTERDAM

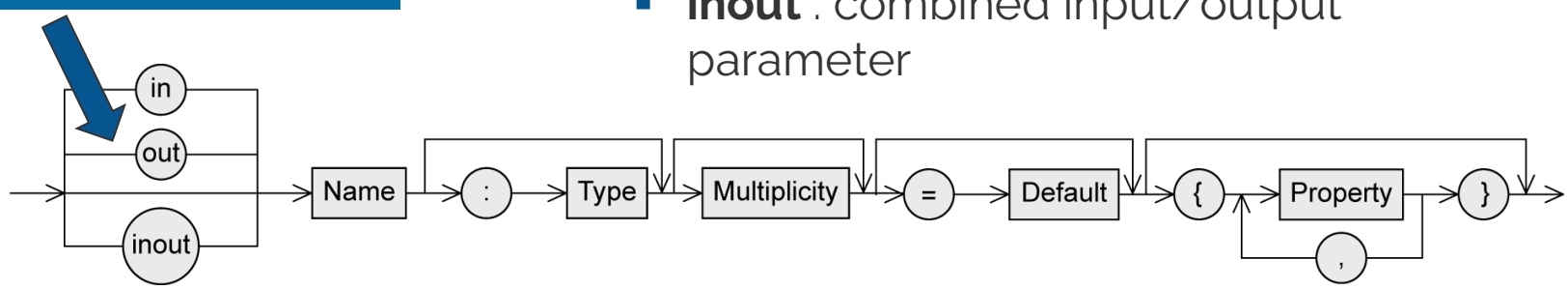
Operation syntax - parameters



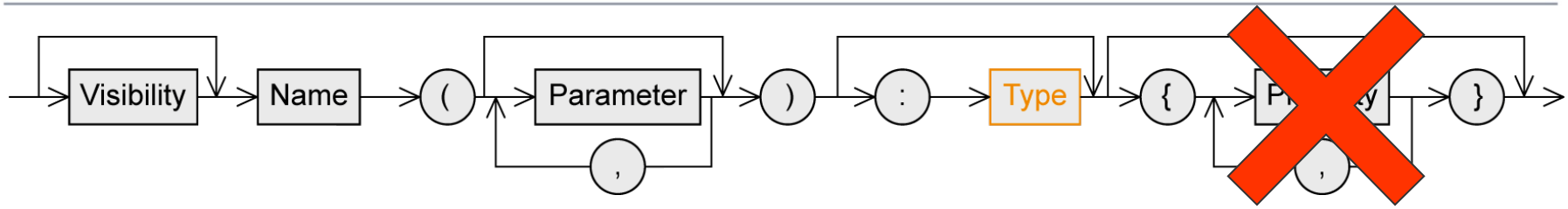
Person
…
+ getName(out fn: String, out ln: String): void + updateLastName(newName: String): boolean + getPersonsNumber(): int

- Notation similar to attributes
- Direction of the parameter
  - **in** … input parameter
    - When the operation is used, a value is expected from this parameter
  - **out** … output parameter
    - After the execution of the operation, the parameter has adopted a new value
  - **inout** : combined input/output parameter

Only situationally useful; suggestion: ignore them



Operation syntax - type



Person
...
getName(out fn: String, out ln: String): void updateLastName(newName: String): boolean getPersonsNumber(): int

- Type of the return value

Properties for operations exist, but they are rarely used (we don't cover them). You can check them out here: https://www.uml-diagrams.org/operation.html

VU VRIJE UNIVERSITEIT AMSTERDAM

Class variables and class operations

- Default: instance variable (= instance attribute): attributes defined on the instance level
- Class variable (= class attribute, static attribute)
  - Defined only once per class,
    - i.e., shared by all instances of the class
    - e.g., counters for the number of instances of a class, constants, etc.
- Class operation (= static operation)
  - Can be used if no instance of the corresponding class was created
    - e.g., constructors, counting operations, math functions, etc.
- Notation: <u>underlining</u> the name of class variable or class operation

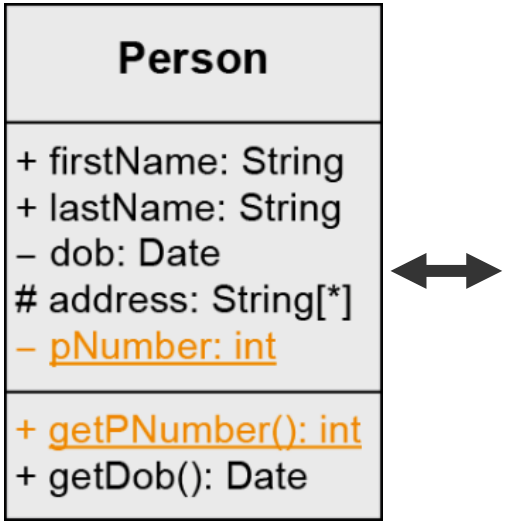
VU VRIJE UNIVERSITEIT AMSTERDAM

Class variables and class operations - example

How would you implement this in Java?

Class variable

Class operation



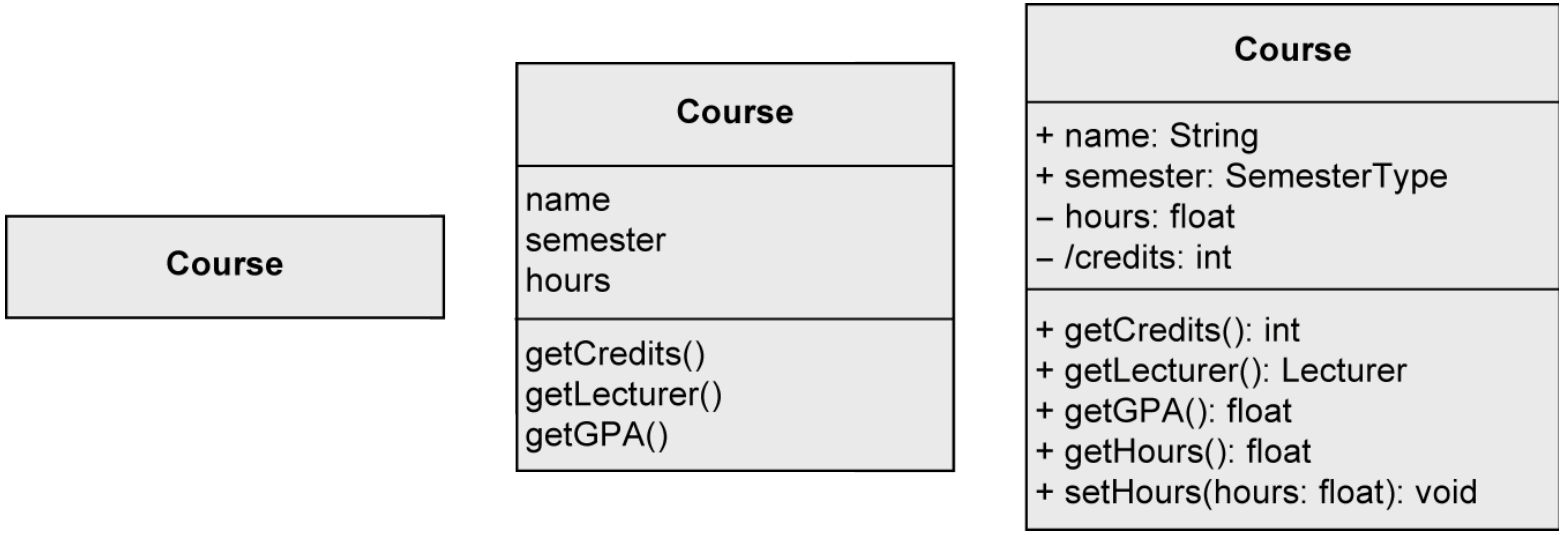

```
class Person {

    public String firstName;
    public String lastName;
    private Date dob;
    protected String[] address;
    private static int pNumber;
    public static int getPNumber() {…}
    public Date getDob() {…}
}
```


VU VRIJE UNIVERSITEIT AMSTERDAM

Specification of classes: different levels of detail

coarse-grained fine-grained



You can use a more coarse-grained class diagram as a descriptive model.

VU VRIJE UNIVERSITEIT AMSTERDAM

Domain classes vs. technical classes [1]

- Domain classes (sometimes called domain entities)
  - Represent concepts from the real-world business domain
  - Follow established domain terminology ("ubiquitous language")
  - Examples: Customer, Order, Product, Player, Payment, etc.
- Technical classes
  - Represent software-related concepts outside the domain
  - Often relate to specific system layers or technical functions
  - Examples: DBConnector, JSONParser, …Repository, …Controller, …Manager, …Validator, etc.
- Both are important but the ratio in a system can vary depending on your design philosophy
- Recommendation: start with the core domain classes and then transition to technical classes to manage them

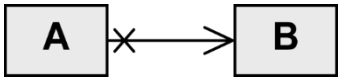
[1] E. Evans, Domain-driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2004.

Relationships Between Classes

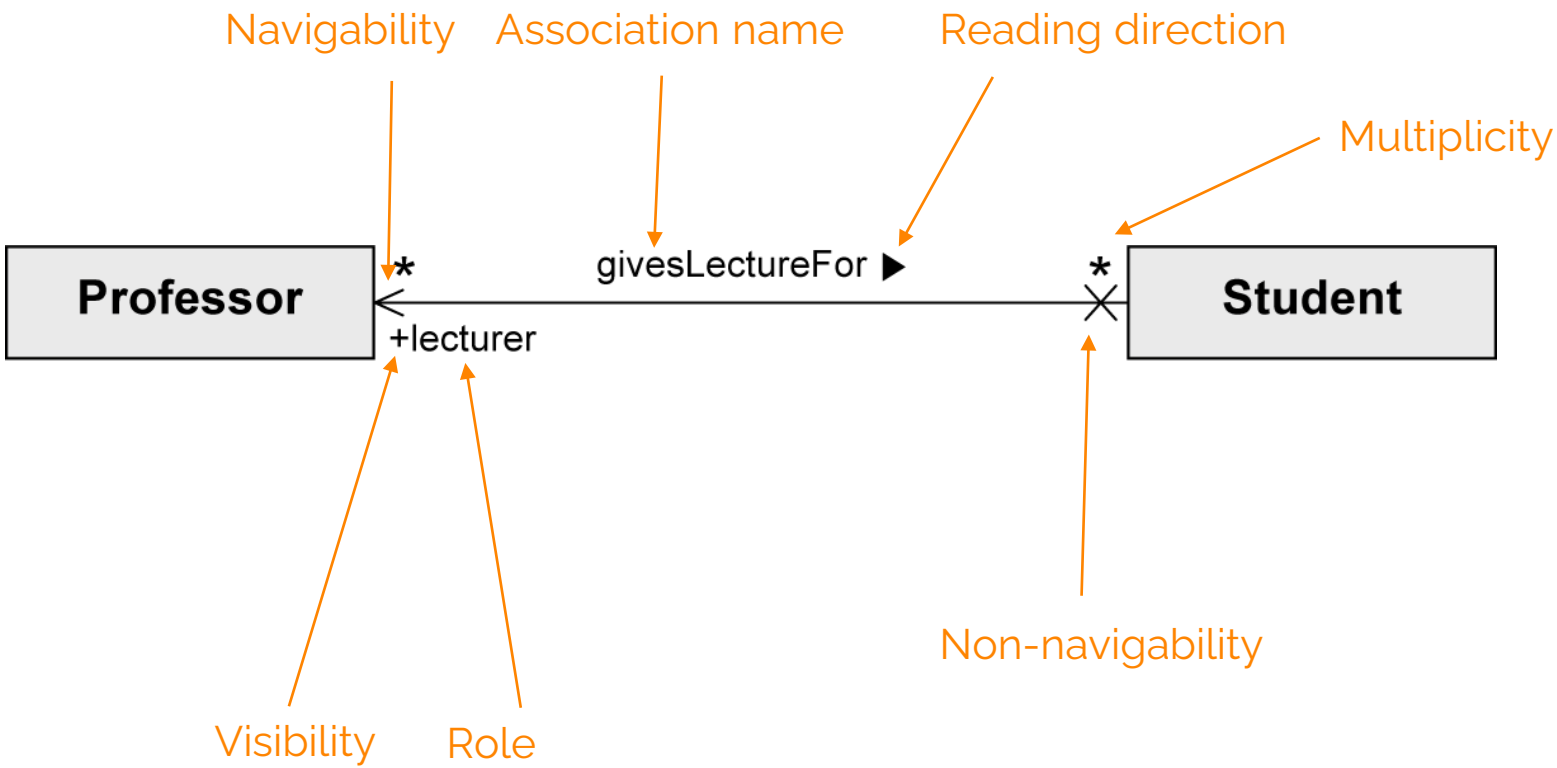
Roadmap

- Associations
  - Binary
  - Association Class
- Aggregation / Composition
- Generalization
  - Inheritance
  - Abstract classes
- «Uses» dependency

Binary association



Connects instances of two classes with one another **via at least one attribute**



VU VRIJE UNIVERSITEIT AMSTERDAM

Binary association - navigability

Navigability: an object knows its partner objects and can therefore access their visible attributes and operations

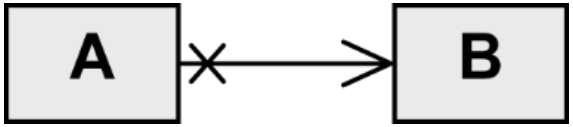
- Indicated by open arrow head

Non-navigability

- Indicated by a cross, but this is also often left out

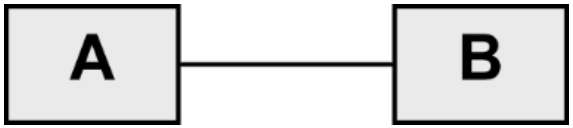
Example:

- **A** can access the visible attributes and operations of **B**
- **B** cannot access any attributes and operations of **A**



Navigability undefined

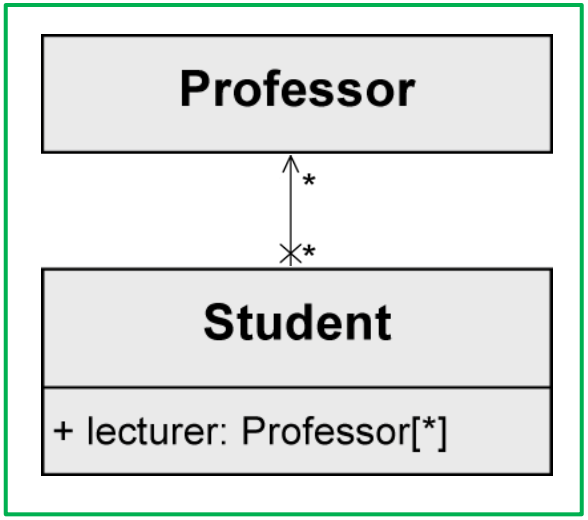
- Bidirectional navigability is assumed

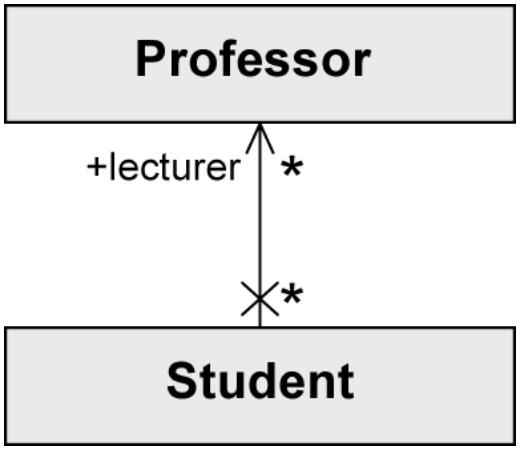


VU VRIJE UNIVERSITEIT AMSTERDAM

Binary association representation

I prefer this one





In Java:

```
class Professor {…}

class Student {
  public Professor[] lecturer;
  …
}
```

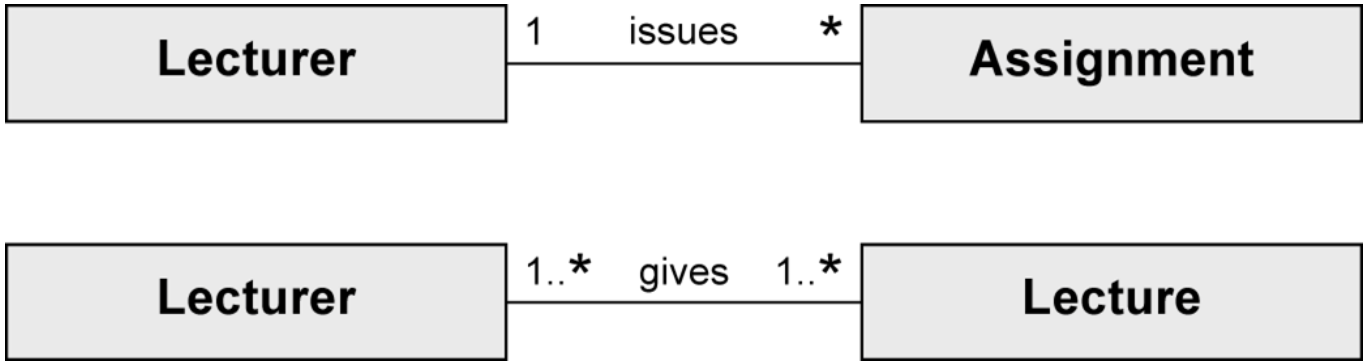
See anything here that is not ideal?

A collection should be reflected in the variable name, e.g., `lecturers` or `lecturerList`.

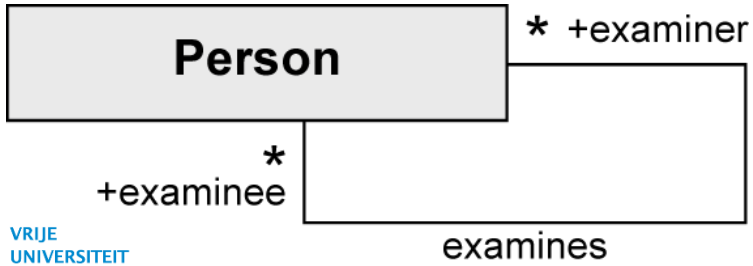
VU VRIJE UNIVERSITEIT AMSTERDAM

Binary association – multiplicity and role

Multiplicity: Number of objects that may be associated with exactly one object of the opposite side

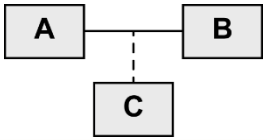


Role: describes the way in which an object is involved in an association relationship

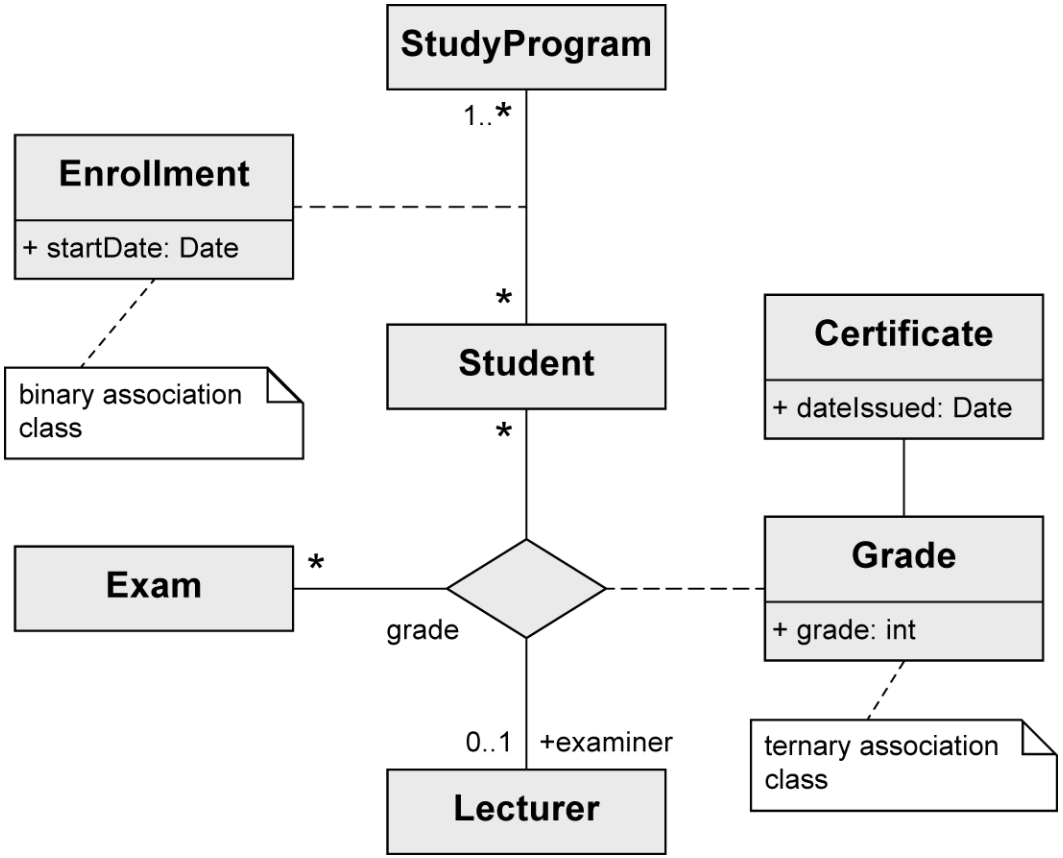



```java
public class Person {
    public ArrayList<Person> examiner;
    public ArrayList<Person> examinee;
}
```

Association class



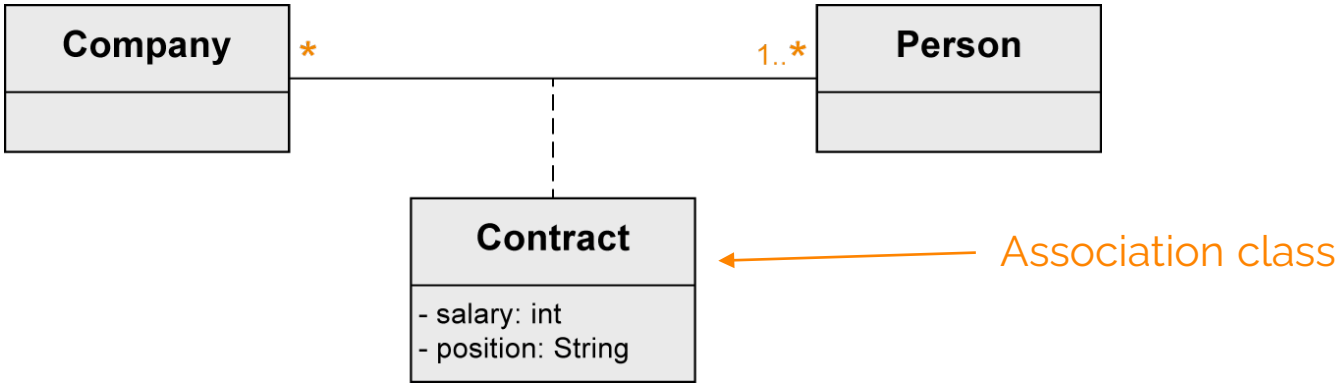
Assign attributes to the relationship between classes rather than to a class itself



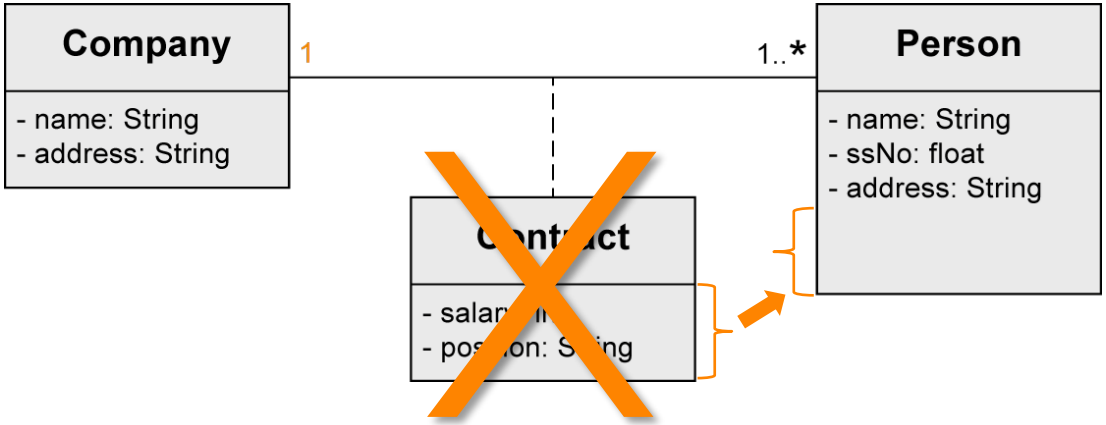
VU VRIJE UNIVERSITEIT AMSTERDAM

When to use an association class

- Mandatory when modeling n:m associations



- With 1:1 or 1:n → possible but not mandatory



VU VRIJE UNIVERSITEIT AMSTERDAM

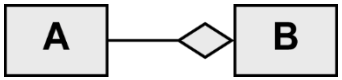
Aggregation

- Special form of association
- Used to express that a class **is part of** another class (whole-part relationship)

- Properties of the aggregation association:
  - **Transitive**: if **B** is part of **A** and **C** is part of **B**, **C** is also part of **A**
  - **Asymmetric**: it is not possible for **A** to be part of **B** and **B** to be part of **A** simultaneously

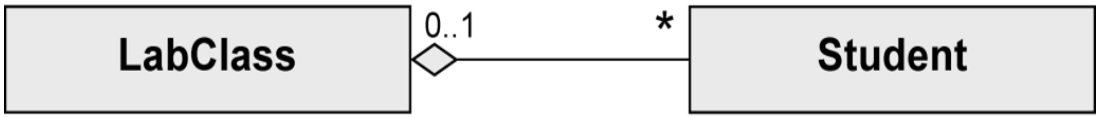
- Two types:
  - (Shared) aggregation
  - Composition

(Shared) Aggregation

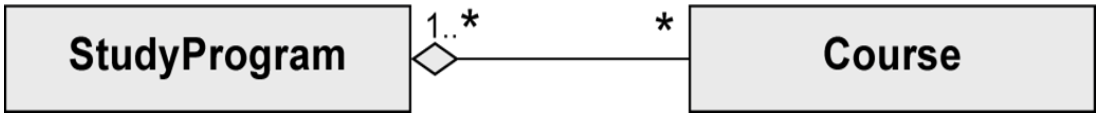


- Expresses a weak belonging of the parts to a whole
  - Parts also exist independently of the whole
- Multiplicity at the aggregating end may be >1
  - One element can be part of multiple other elements simultaneously

- Example:
  - **Student** is part of **LabClass**



  - **Course** is part of **StudyProgram**



VU VRIJE UNIVERSITEIT AMSTERDAM

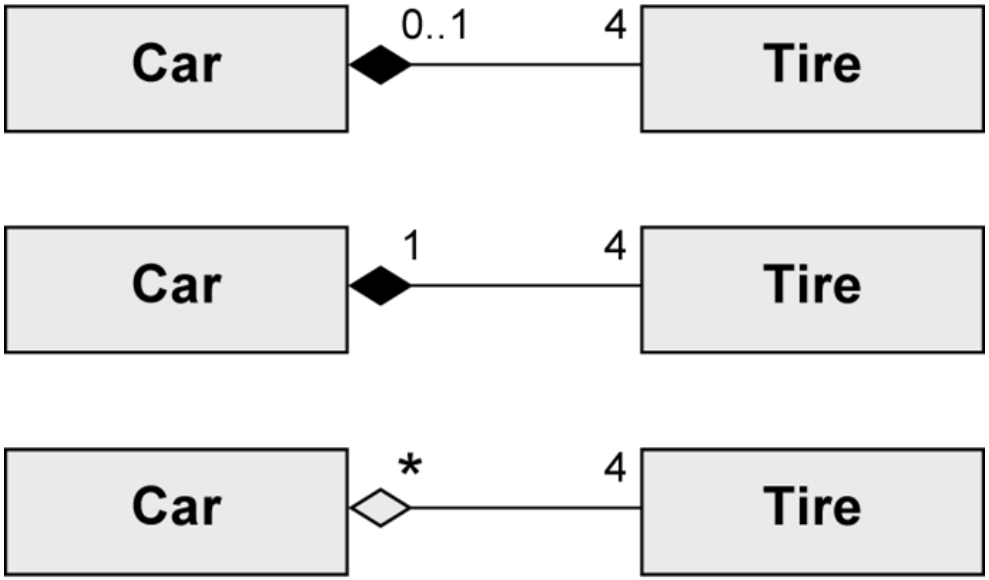
Composition

- **Existence dependency** between the composite object and its parts
- An existing part is contained in at most one composite object at one specific point in time
  - Multiplicity at the aggregating end max 1

- If the composite object is deleted, its parts are also deleted → the part **usually** cannot exist without the whole (exceptions are possible with multiplicity `0..1` instead of `1`)

Aggregation vs. composition

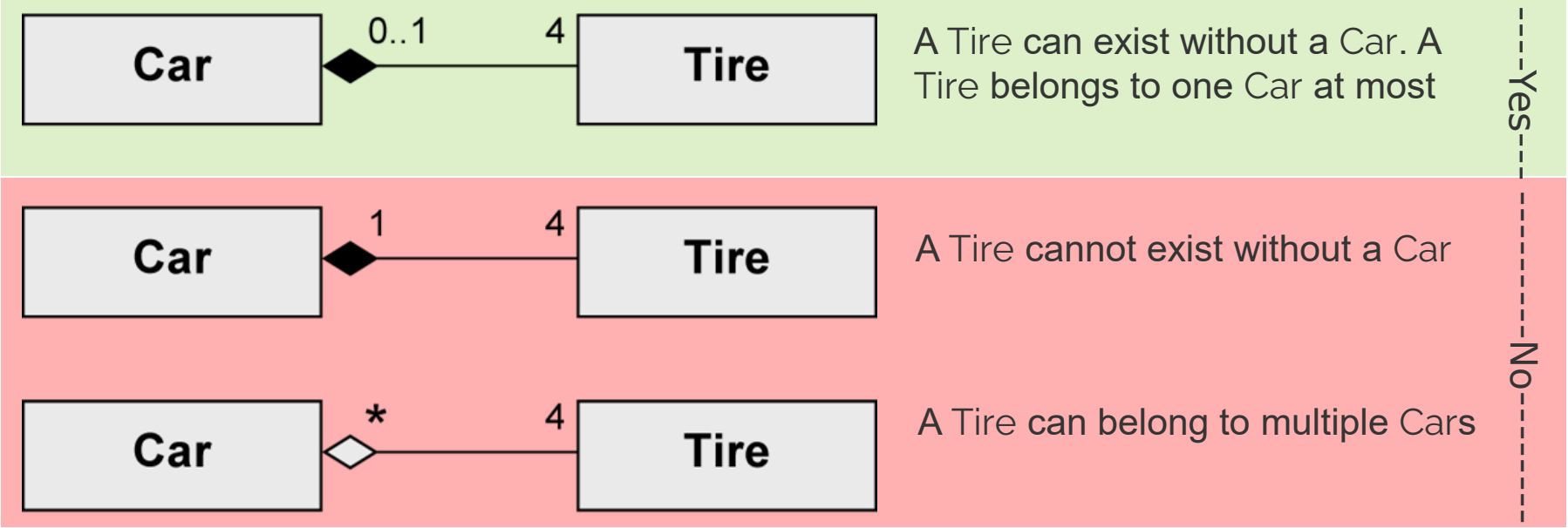
Which one is the most reasonable representation of reality?



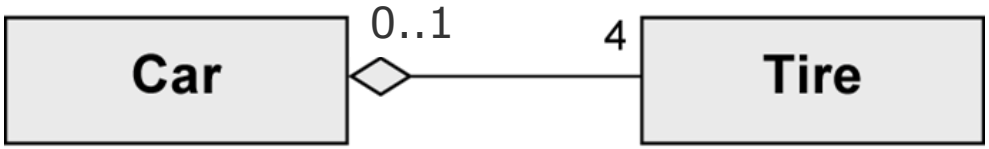
VU VRIJE UNIVERSITEIT AMSTERDAM

Aggregation vs. composition

Which one is the most reasonable representation of reality?

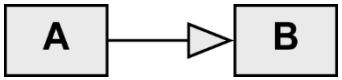


Another valid option:

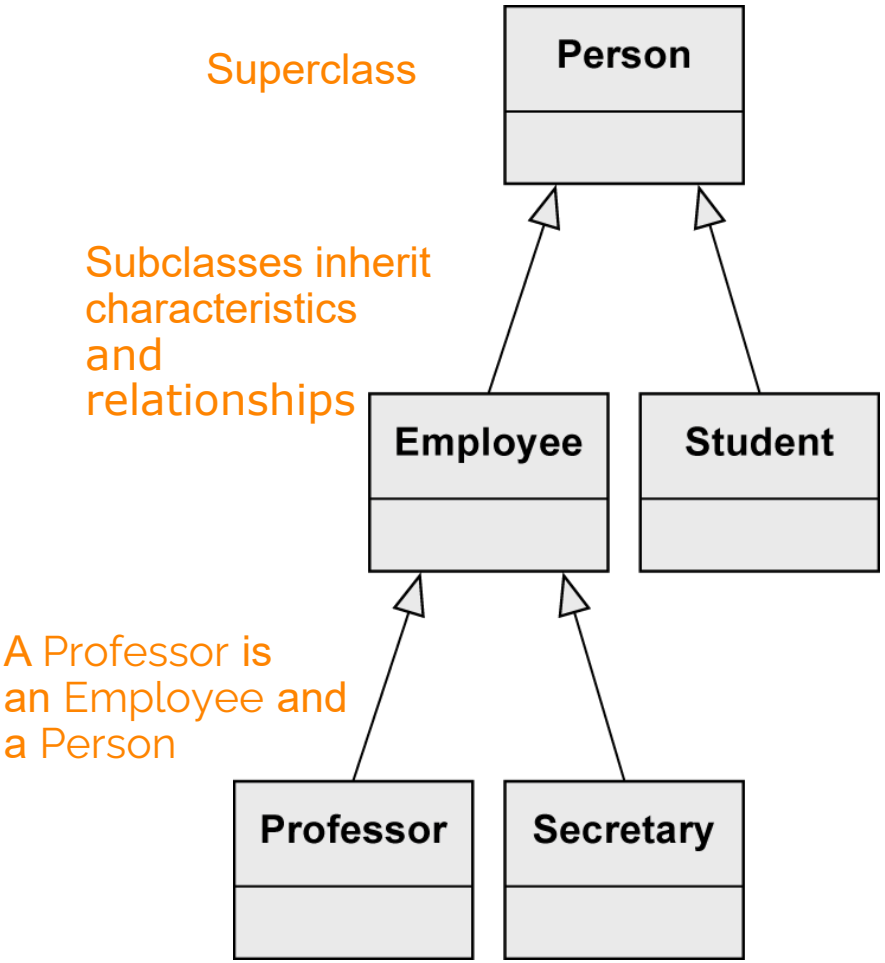


VU VRIJE UNIVERSITEIT AMSTERDAM

Generalization - inheritance

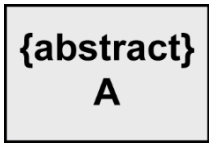


- Attributes, operations, and relationships of the general class **are passed on to its subclasses** (except private ones)
- Every instance of a subclass is also an indirect instance of the superclass
- Subclasses may have further characteristics and relationships
- Generalizations are transitive

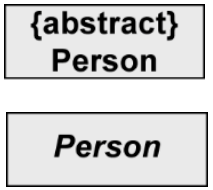


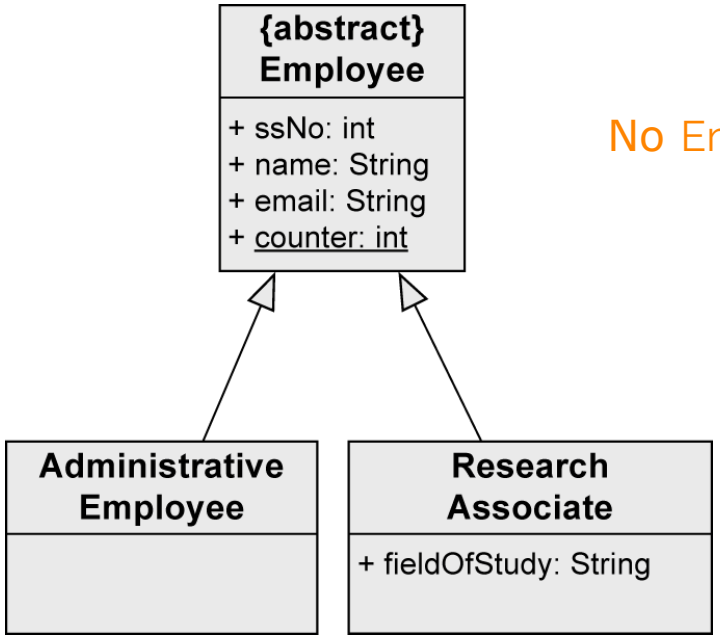
VU VRIJE UNIVERSITEIT AMSTERDAM

Generalization – abstract classes



- Used to highlight common characteristics of their subclasses while ensuring **no direct instances of the superclass**
- Only its non-abstract subclasses can be instantiated
- Useful in the context of generalization relationships



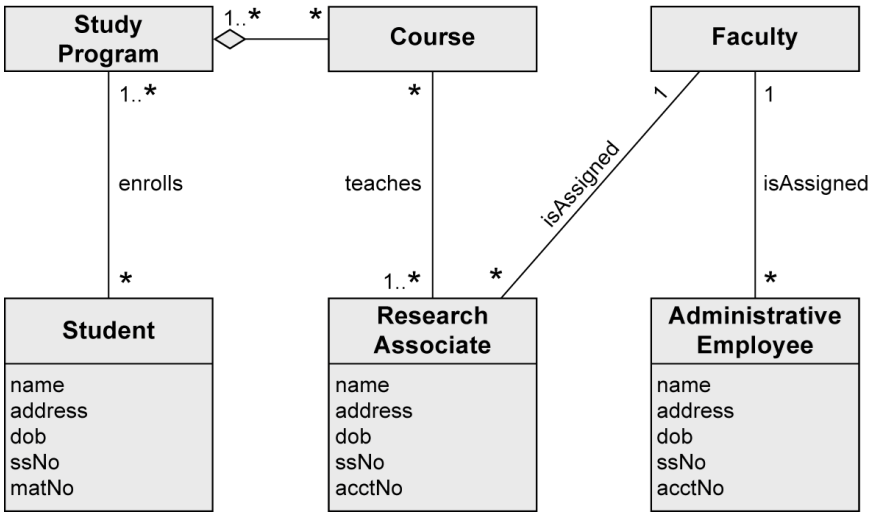


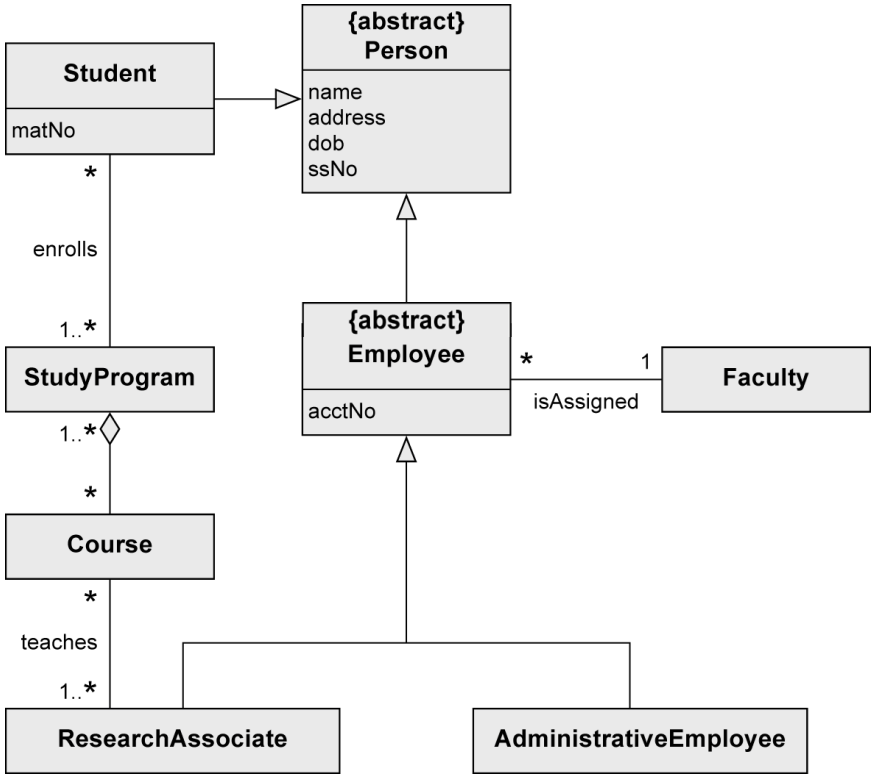
No Employee object possible

Two types of Employees: Administrative Employee and Research Associate

VU VRIJE UNIVERSITEIT AMSTERDAM

Example: generalization avoids duplication

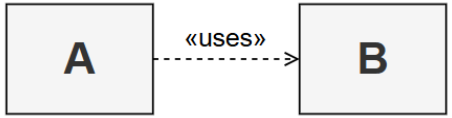




Do you see something "incorrect" here?

VU VRIJE UNIVERSITEIT AMSTERDAM

«Uses» dependency [1]



- Indicates that one class depends on another class without having an explicit attribute of that class type
- One class requires the other for its full implementation
- Typical examples:
  - Class B is used as a method parameter within A
  - Class B is used as return type of a method of A
  - Class B is temporarily instantiated inside a method of A (sometimes also indicated via «creates»)
  - Class A temporarily retrieves an instance of B from somewhere
- Appears often between technical and domain classes

[1] https://www.uml-diagrams.org/dependency.html

Relationship decision guidance

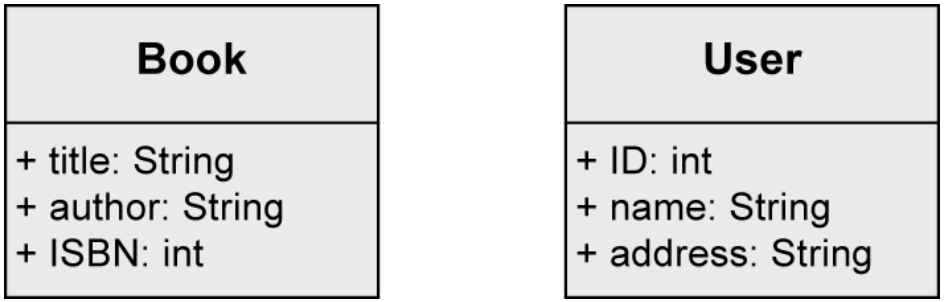
1. Is one class a **specialized version** of the other, e.g., person and employee?
   Yes: **Generalization**, No: continue
2. Does one class have an **attribute of the other class type**?
   No: **«Uses» dependency**, Yes: continue
3. Is one class a **part** of the other (whole-part relationship), e.g., institute and faculty or CPU and computer?
   No: **Association**, Yes: continue
4. Is the whole-part relationship **very strong** (part cannot really exist without whole, destroying the whole also destroys parts, part can only belong to at most one whole)?
   Yes: **Composition**, No: **Aggregation**

Creating Class Diagrams (Concrete Example)

Creating a class diagram

- Difficult to extract classes, attributes, and associations from natural language automatically
- Guidelines
  - **Nouns** often indicate classes, but can also be attributes ("have")
  - **Adjectives** often indicate attribute values
  - **Verbs** often indicate operations or relationships

- Example: the library management system stores users with their unique ID, name, and address as well as books with their title, author, and ISBN number. Ann Foster wants to use the library.



Question: What about "Ann Foster"?

VU VRIJE UNIVERSITEIT AMSTERDAM

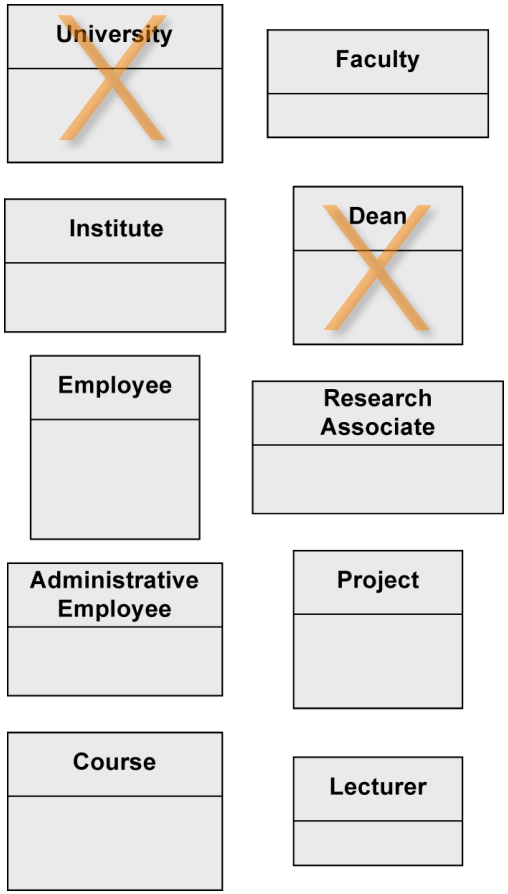
Example – University Information System

- A university consists of multiple faculties, which are composed of various institutes.
- Each faculty and each institute has a name.
- An address is known for each institute.
- Each faculty is led by a dean, who is an employee of the university.
- The total number of employees is known.
- Employees have a social security number, a name, and an email address. There is a distinction between research and administrative personnel.
- Research associates are assigned to at least one institute. The field of study of each research associate is known. Furthermore, research associates can be involved in projects for a certain number of hours, and the name, starting date, and end date of the projects are known.
- Some research associates hold courses. Then they are called lecturers.
- Courses have a unique number (ID), a name, and a weekly duration in hours.

Step 1: identifying classes

- A university consists of multiple faculties, which are composed of various institutes.
- Each faculty and each institute has a name.
- An address is known for each institute.
- Each faculty is led by a dean, who is an employee of the university.
- The total number of employees is known.
- Employees have a social security number, a name, and an email address. There is a distinction between research and administrative personnel.
- Research associates are assigned to at least one institute. The field of study of each research associate is known. Furthermore, research associates can be involved in projects for a certain number of hours, and the name, starting date, and end date of the projects are known.
- Some research associates hold courses. Then they are called lecturers.
- Courses have a unique number (ID), a name, and a weekly duration in hours.

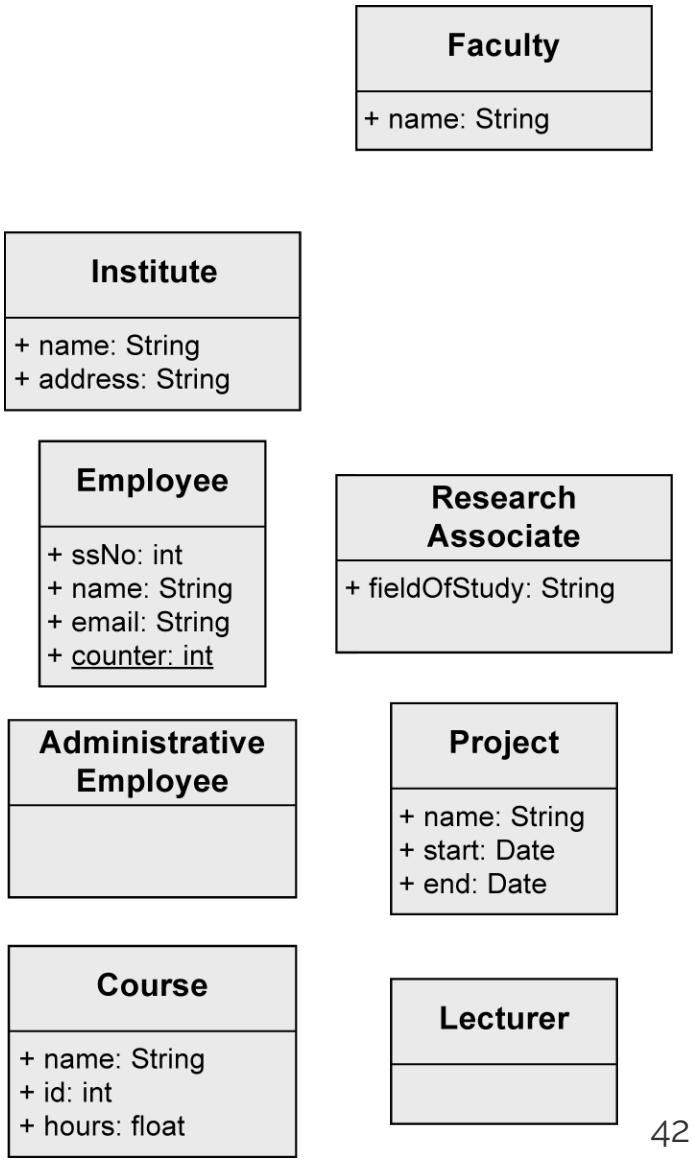
We model the system „University“



Dean has no further attributes than any other employee

Step 2: identifying attributes

- A university consists of multiple faculties which are composed of various institutes.
- Each faculty and each institute has a name.
- An address is known for each institute.
- Each faculty is led by a dean, who is an employee of the university.
- The total number of employees is known.
- Employees have a social security number, a name, and an email address. There is a distinction between research and administrative personnel.
- Research associates are assigned to at least one institute. The field of study of each research associate is known. Furthermore, research associates can be involved in projects for a certain number of hours, and the name, starting date, and end date of the projects are known.
- Some research associates hold courses. Then they are called lecturers.
- Courses have a unique number (ID), a name, and a weekly duration in hours.

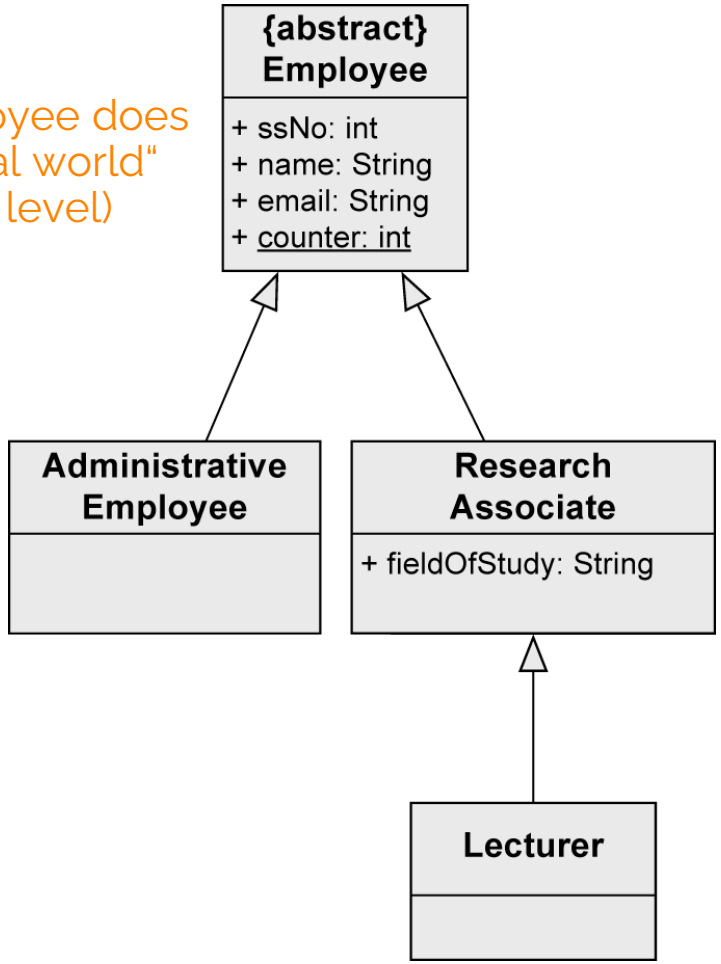


Step 3: identifying relationships (1/6)

Typical relationships between domain classes:

- Association
- Generalization
- Aggregation

Abstract, i.e., employee does not exist in the „real world“ (i.e., at the instance level)



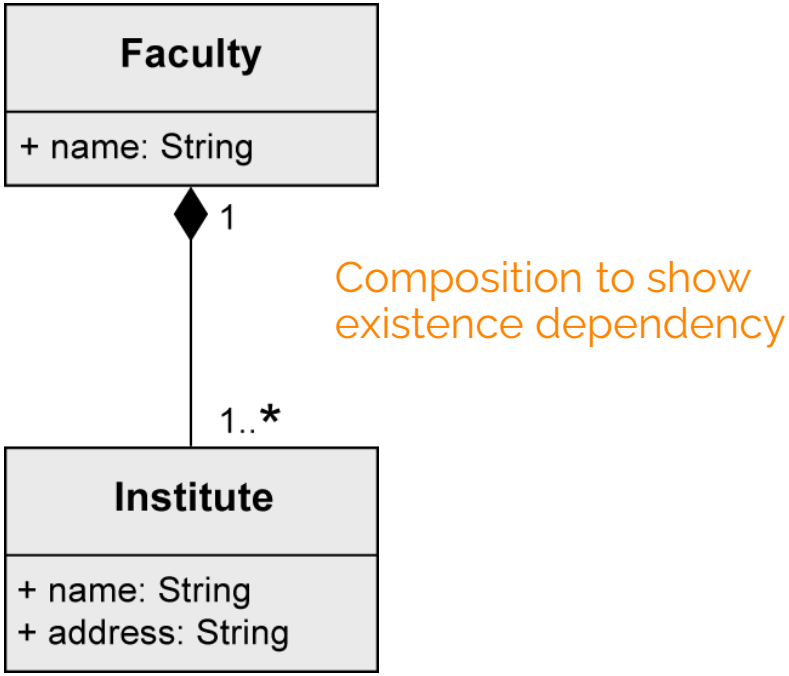
Indication of a generalization

- *“There is a distinction between research and administrative personnel.”*
- *“Some research associates hold courses. Then they are called lecturers.”*

VU VRIJE UNIVERSITEIT AMSTERDAM

Step 3: identifying relationships (2/6)

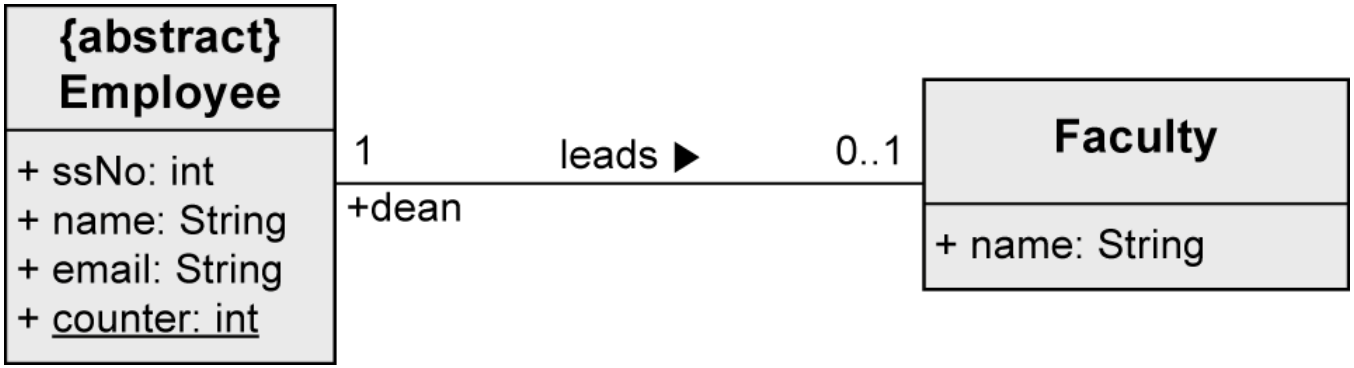
"A university consists of multiple faculties which are composed of various institutes."



VU VRIJE UNIVERSITEIT AMSTERDAM

Step 3: identifying relationships (3/6)

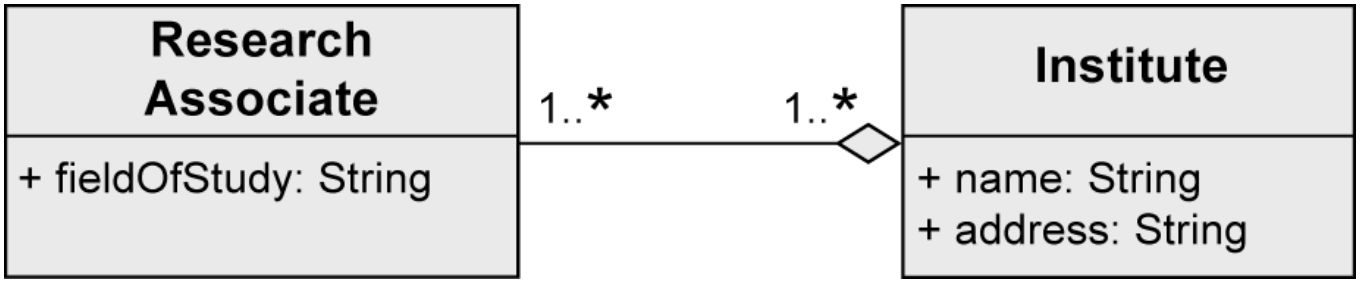
"Each faculty is led by a dean, who is an employee of the university"



In the `leads` relationship, the `Employee` takes the role of a dean.

Step 3: identifying relationships (4/6)

"Research associates are assigned to at least one institute."

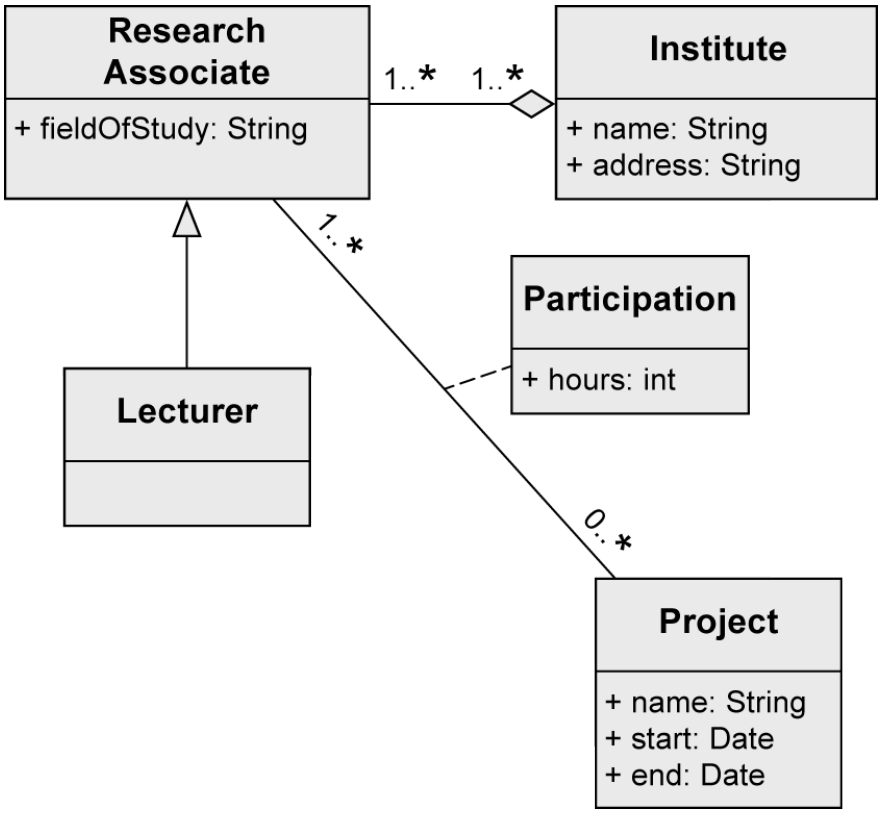


Aggregation to show that `ResearchAssociates` are part of an `Institute`, but there is no existence dependency

VU VRIJE UNIVERSITEIT AMSTERDAM

Step 3: identifying relationships (5/6)

"Furthermore, research associates can be involved in projects for a certain number of hours."

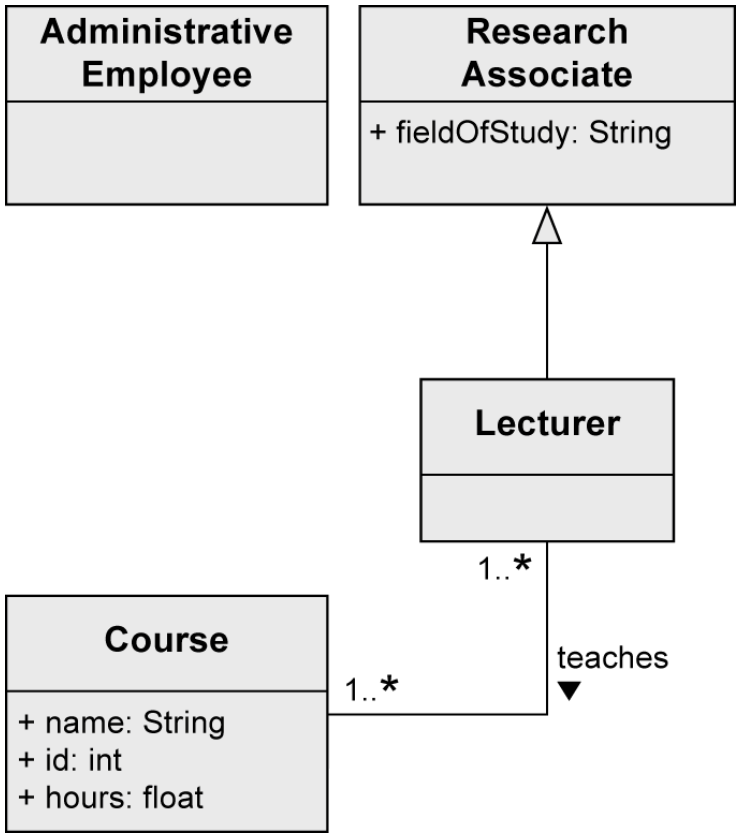


Association class enables to store the number of hours for every `Project` of every `ResearchAssociate`

VU VRIJE UNIVERSITEIT AMSTERDAM

Step 3: identifying relationships (6/6)

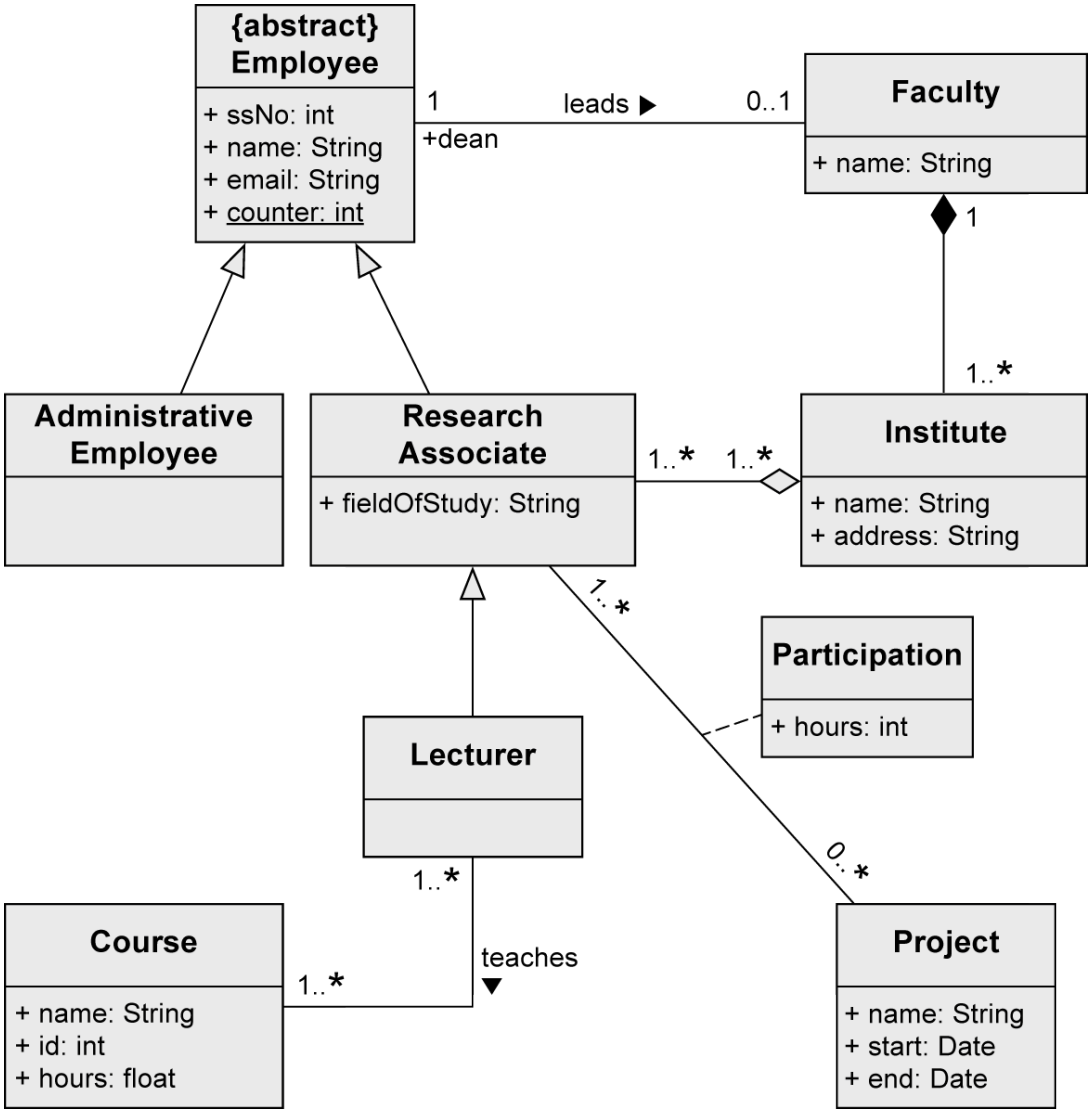
"Some research associates hold courses. Then they are called lecturers."



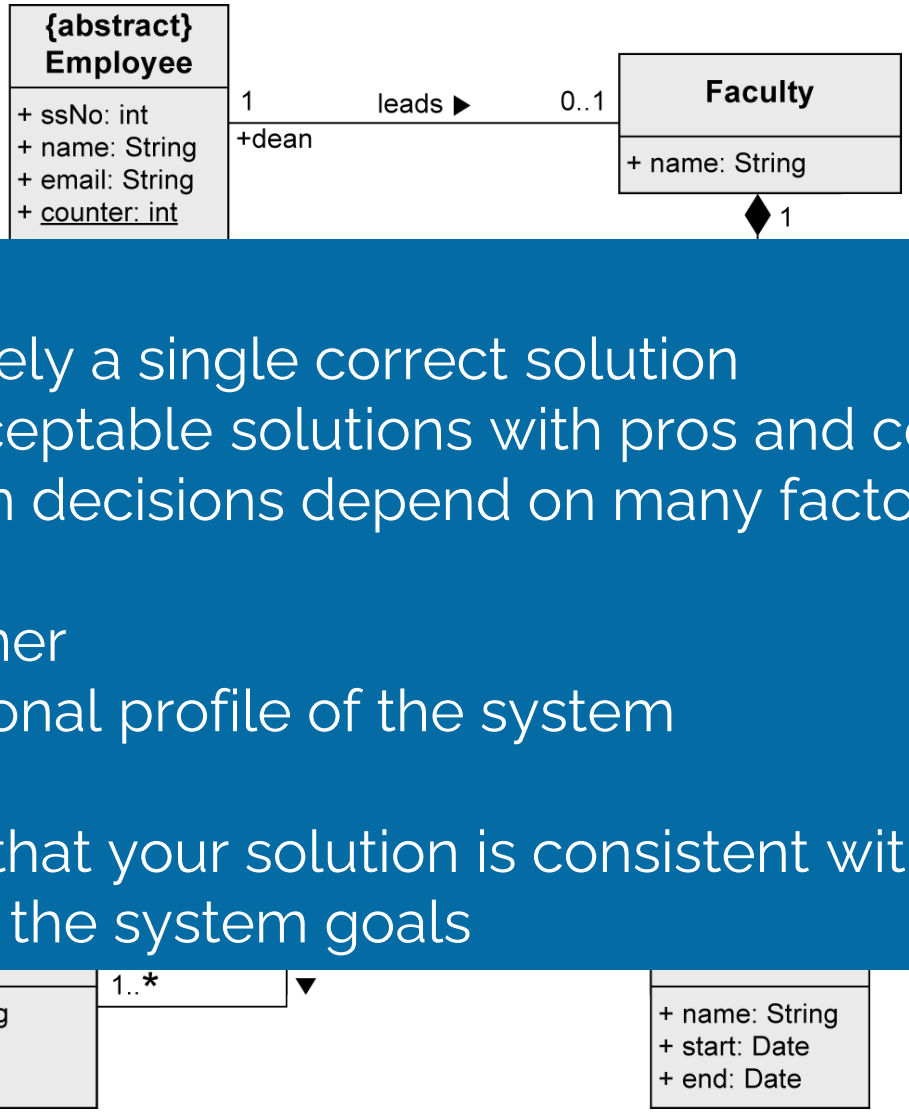
- `Lecturer` inherits all characteristics, associations, and aggregations from `ResearchAssociate`
- `Lecturer` has a `teaches` association with `Course`

VU VRIJE UNIVERSITEIT AMSTERDAM

Complete class diagram



Complete class diagram



REFLECTION

- There is rarely a single correct solution
- Several acceptable solutions with pros and cons
- Your design decisions depend on many factors, like:
  - Intent
  - Consumer
  - Operational profile of the system
  - …
- Make sure that your solution is consistent with these factors and the system goals

VU VRIJE UNIVERSITEIT AMSTERDAM